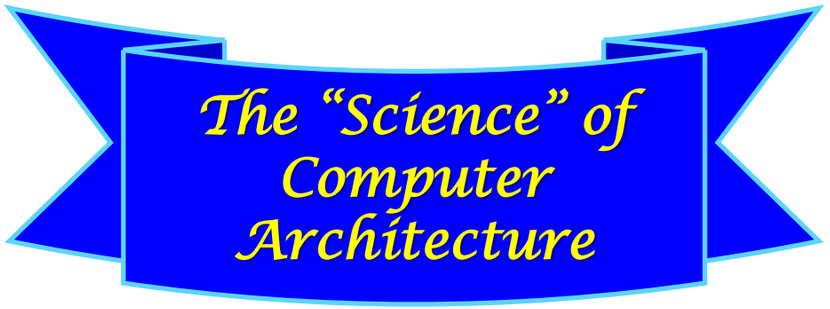
18-740: Modern Computer Architecture & Design

Lecture 12: "Performance-Power-Energy Efficiency Iron Laws"

John Paul Shen
October 01, 2026



- Recommended References:
  - "Energy per Instruction Trends in Intel® Microprocessors," by Ed Grochowski, Murali Annavaram, 2006.
  - "Mitigating Amdahl's Law through EPI Throttling," by M. Annavaram, E. Grochowski, J. P. Shen, In 32nd ISCA 2005.

Electrical & Computer ENGINEERING

18-740: Modern Computer Architecture & Design

Evolution of Von Neumann Implementations:

- ~~SP: Sequential Processors (direct implementation of sequential execution)~~
- PP: Pipelined Processors (overlapped execution of in-order instructions)
- SSP: Superscalar Processors (out-of-order execution of multiple instructions)
- MCP: Multi-core (Asym.) Processors (concurrent multi-threads and/or multi-programs)
- HMP: Heterogeneous Multi-Processors (concurrent multi-tasks on diverse co-processors)

18-740

Parallelisms for **Performance** (→ for **Power** Reduction → for **Energy** Efficiency)

- **ILP**: Basic Block/Loop Iteration (exploit ILP in PP, SSP, VLIW)
- **DLP**: Data Set (exploit DLP in VP/SIMD, GPU)
- **TLP**: Thread/Task (exploit TLP in MCP, HMP)
- **SLP**: System/Workload (exploit SLP in MCC: Multi-Computer Clusters)

18-740: Modern Computer Architecture & Design

Lecture 15: "Performance-Power-Energy_Efficiency Iron Laws"

A. Law #1 – **Latency** (Single-Thread) Performance (ILP)
B. Law #2 – **Throughput** (Multi-Thread) Performance (TLP)
C. Law #3 – Throughput Performance **Scalability** (new)
D. Law #4 – Performance Scalability **Impediments**
E. Law #5 – **Performance** and **Power** Scaling
F. Law #6 – **Power** and **Energy** Optimizations



Electrical & Computer ENGINEERING

Law #1 – Latency (Single-Thread) Performance

- Time to execute a program: T (latency)

$$T = \frac{instructions}{program} \times \frac{cycles}{instruction} \times \frac{time}{cycle}$$

$$T = PathLength \times CPI \times CycleTime$$

- Latency (ST) performance: Perf = 1/T

$$Perf_{ST} = \frac{1}{PathLength \times CPI \times CycleTime} = \frac{Frequency}{PathLength \times CPI}$$

Two Classes of Computer Workloads

Scalar workloads	**Scalar uarch (ILP)**
• IPC 0.2 – 2 • Examples: software development tools, office productivity suites, OS kernel routines	• Reduce effective execution latency! • Uarch techniques: speculative execution, branch prediction, caching • Example: fastest biggest OOO core (SISD) • High energy/instruction
Parallel workloads	**Parallel uarch (TLP)**
• IPC of 4 to several thousand • Examples: 3D graphics, media processing, scientific applications • Utilize instruction-level parallelism and thread-level parallelism	• Provide large execution bandwidth! • Uarch techniques: wide superscalar, multicores, multiprocessing, multithreading, • Examples: many small efficient cores (SIMD,MIMD) • Low energy/instruction

- Multithreaded programs contain both sequential and parallel phases!
  - Want to minimize run-time for both sequential and parallel phases
  - **Amdahl's law**: speedup via parallelization will be limited by sequential component

Latency (ST) vs. Throughput (MT) Performance

- **Reduce Latency of Application**
  - Uni-Processor (1), Single Program
  - Target Single-Thread (ST) Performance
  - Examples: SPEC, PC and Workstations
- **Increase Throughput of System**
  - Multiple Processors (n), Many Threads (MT)
  - Multi-Core Processors (MCP), Multiple MCP's
  - Target Multi-threaded/Multi-tasking Throughput
  - Example: Database Transaction Processing

Ideal Throughput (Multi-Thread) Performance

- Time to process a thread/task: T (latency)

$$T = \frac{instructions}{thread} \times \frac{cycles}{instruction} \times \frac{time}{cycle}$$

$$T = PathLength \times CPI \times CycleTime$$

- Throughput (MT) performance: Perf = n x 1/T

$$Perf_{MT} = \frac{n \times 1}{PathLength \times CPI \times CycleTime} = \frac{n \times Frequency}{PathLength \times CPI}$$

Law #2 – Throughput (Multi-Thread) Performance

- Throughput (MT) Performance:

$$Perf_{MT} = \frac{n \times Frequency}{PL(n) \times CPI(n)}$$

- Can Improve $Perf_{MT}$ by:
  - Increasing: n (no. of CPUs or cores)
  - Increasing: $Frequency$ (CPU clock frequency)
  - Decreasing: PL (dynamic instruction count)
  - Decreasing: CPI (cycles/instruction)

A New Throughput Performance Scalability Model

- Throughput (MT) Performance Speedup:

$$Speedup(n)_{MT} = \frac{\frac{n \times Frequency}{PL(n) \times CPI(n)}}{\frac{Frequency}{PL(1) \times CPI(1)}} = n \times \left(\frac{PL_1 \times CPI_1}{PL(n) \times CPI(n)} \right)$$

- A Rigorous Scalability Model (new):

$$PL(n) \times CPI(n) \cong n^x PL_1 \times n^y CPI_1$$

Law #3 – Throughput Performance Scalability

- Throughput (MT) Performance Speedup:

$$Speedup(n)_{MT} = n \times \left(\frac{PL_1 \times CPI_1}{n^x PL_1 \times n^y CPI_1} \right) = \left(\frac{n}{n^x \times n^y} \right)$$

- Scalability Impedance Functions:

$$\begin{cases} n^x \equiv PL(n)\ Impedance\ Function \\ n^y \equiv CPI(n)\ Impedance\ Function \end{cases}$$

$$\left(n^x \times n^y\right) = n^{(x+y)} \;\Bigg|\; \begin{cases} (x+y) = 1.0 \Rightarrow No\ Speedup \\ (x+y) = 0.0 \Rightarrow No\ Impedance \end{cases}$$

Real World Example: OLTP Database Performance

- Throughput Database Performance: TPS

$$TPS = \frac{Transactions}{Second} = \frac{n \times Frequency}{IPX(n) \times CPI(n)}$$

[Law #2]

- Can Improve TPS by:
  - Increasing: n (no. of CPUs)
  - Increasing: $Frequency$ (CPU clock frequency)
  - Decreasing: IPX (instructions/transaction) == PL
  - Decreasing: CPI (cycles/instruction)

[Rich Hankins & Murali Annavaram, 2004]

4-way Multiprocessor Experimental Setup





Component	Intel® Xeon™ System	Intel® Itanium® 2 System
Processors	4-way SMP, 1.6GHz	4-way SMP, 900MHz
Caches	256KB L2, 1MB L3	256KB L2, 3MB L3
Operating System	Red Hat® AS 2.1	Red Hat® AS 2.1
Disks	24 data + 2 log	32 data + 1 log
Main Memory	4GB	16GB
Database	Oracle® 9ir2	Oracle® 10g
OS Large Page Size	4MB	256MB
SGA	3GB	14GB

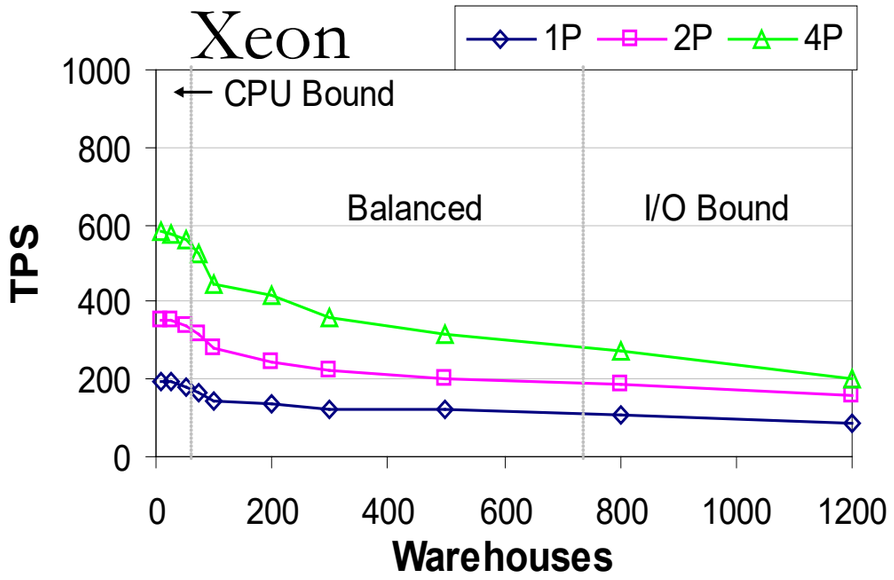
- Based on EMON Events
  - Separate User and OS components for each event
  - Use multiple long runs (20-min warm up, 10-min measurement)
  - Strive for standard deviation <5% for TPS & CPU utilization > 90%
  - Overall user execution time ~70-90%

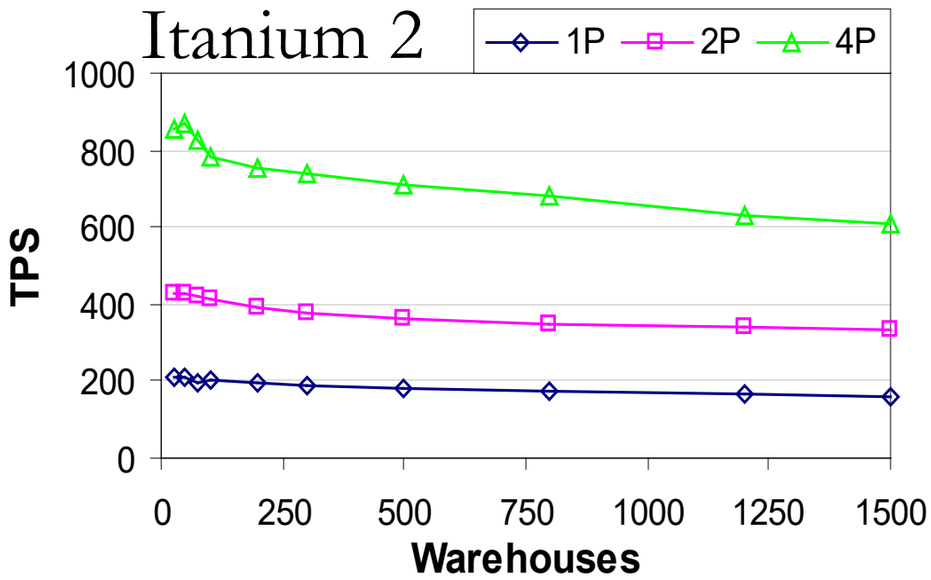
$$TPS = \frac{n \times Frequency}{IPX(n) \times CPI(n)}$$

[Rich Hankins & Murali Annavaram, 2004]

TPS: Throughput Scaling

$$TPS = \frac{n \times Frequency}{IPX(n) \times CPI(n)}$$





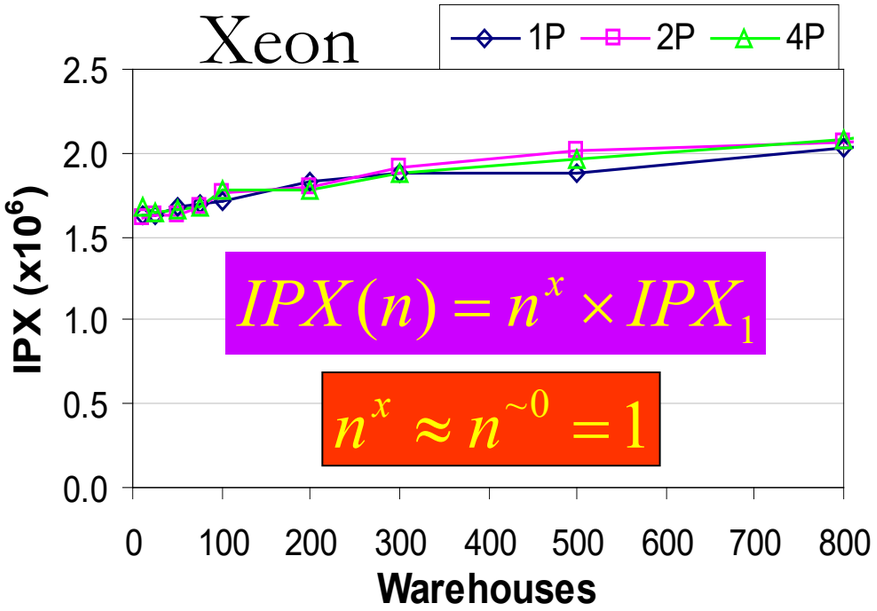
- TPS scales more linearly on Itanium2
  - Larger SGA implies slower I/O rate increase (Xeon I/O rate increases at 7KB/WH, & Itanium 2 at: 6KB/WH)
  - Bus utilization on Xeon higher than Itanium 2 (45% vs. 39%)
- Increasing I/O rate → more processes & context switches
  - Clients increase from 8 to 56 on Xeon & 4 to 64 on Itanium 2
  - OS time up to 20% on Xeon & only 10% on Itanium 2

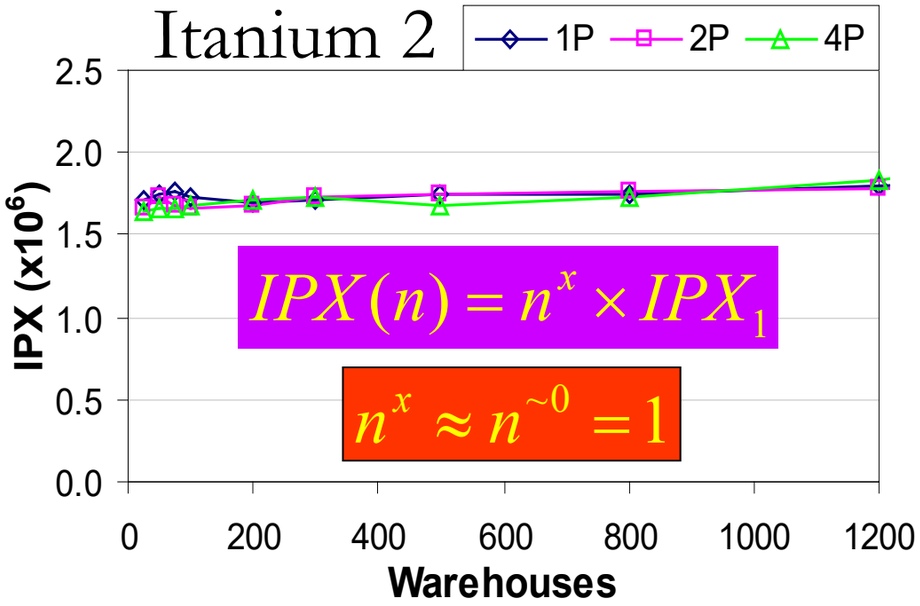
- Performance degrades with increased data set size (due to I/O rate)
- Performance improves with increased n (on IPF almost linear increase)

[Rich Hankins & Murali Annavaram, 2004]

IPX: Path-Length Scaling

$$TPS = \frac{n \times Frequency}{IPX(n) \times CPI(n)}$$





- Growth in IPX (quite linear) attributed to OS IPX increase
  - More I/O, more context switching
- User level IPX remains relatively constant in both systems
  - Code path through Oracle relatively constant
- Excluding NOPS, IPX at 25 WH similar for both systems!
- IPX growth less pronounced on Itanium 2

- IPX increases with increased data set size (Xeon)
- But IPX(n) doesn't increase much with increased n

Path-Length Contributions

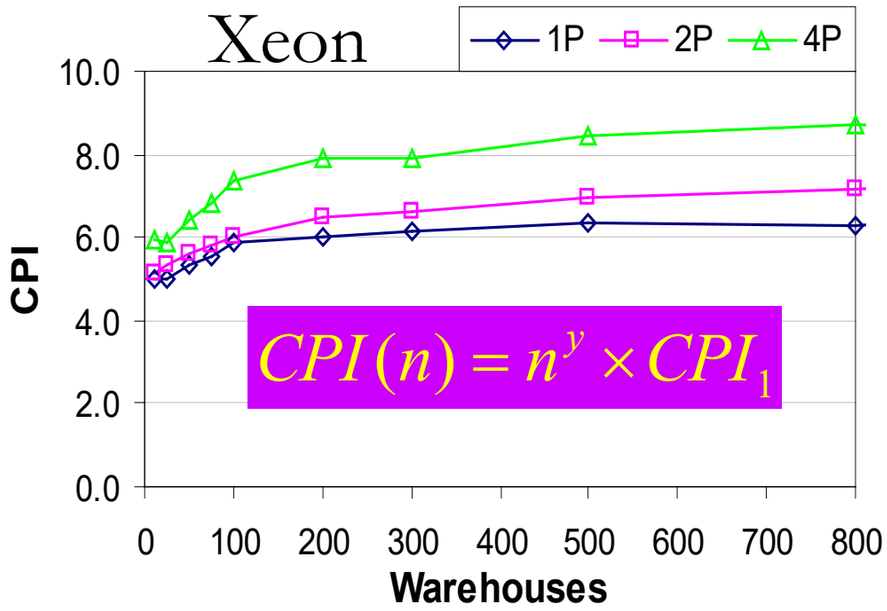
$$TPS = \frac{n \times Frequency}{IPX(n) \times CPI(n)}$$

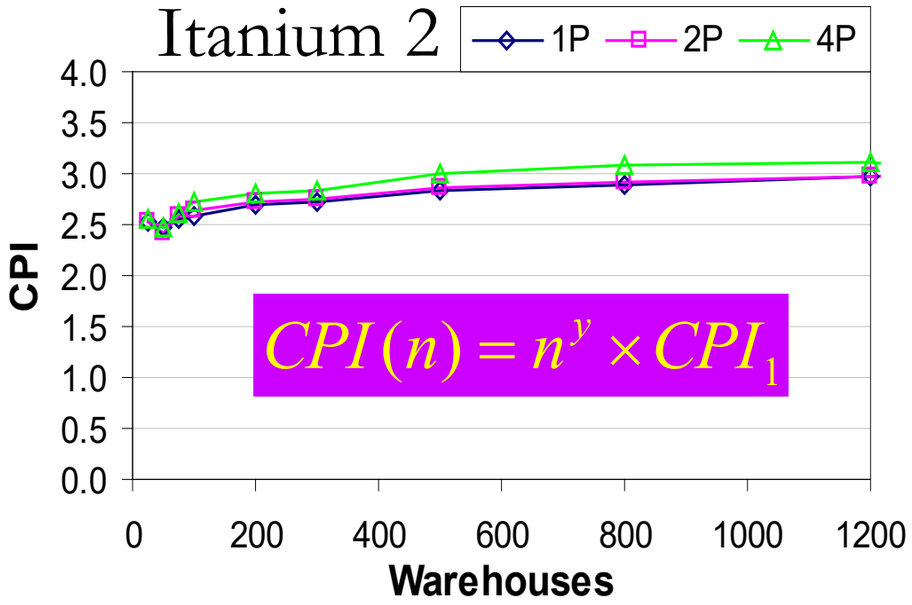
$$PL(n) \begin{cases} OS \begin{cases} I/O \\ Page\ Fault \\ Context\ Switch \end{cases} \\ APP \begin{cases} Original\ Code \\ Speculation \\ Replay \\ Synchronization \end{cases} \end{cases}$$

[Rich Hankins & Murali Annavaram, 2004]

CPI: Overhead Scaling

$$TPS = \frac{n \times Frequency}{IPX(n) \times CPI(n)}$$





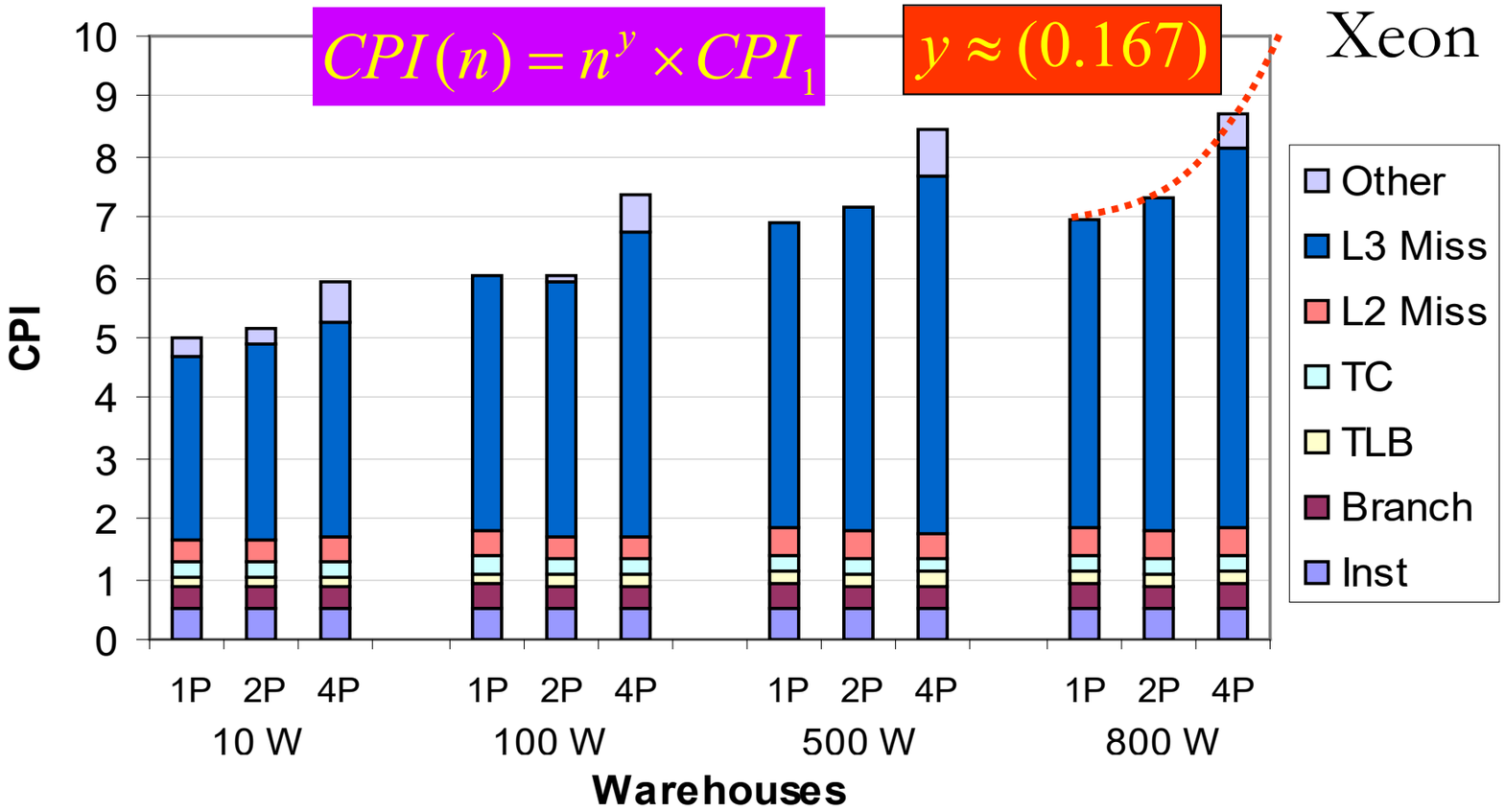
- CPI increases with a "knee" but less sharp on Itanium 2 !
- Overall CPI trend strongly determined by user CPI
  - User mode execution time more than 90% on IPF, 80% on Xeon
- Xeon CPI grows with P, Itanium 2 CPI does not
  - Growth attributed to higher bus utilization on Xeon

- CPI does increase with increased data set size
- CPI(n) also increases with increased n (esp. for Xeon)

[Rich Hankins & Murali Annavaram, 2004]

CPI Breakdown: Xeon

$$TPS = \frac{n \times Frequency}{IPX(n) \times CPI(n)}$$



$$TPS = \frac{n \times Frequency}{\left(n^0 \times IPX_1\right) \times \left(n^{0.167} \times CPI_1\right)}$$

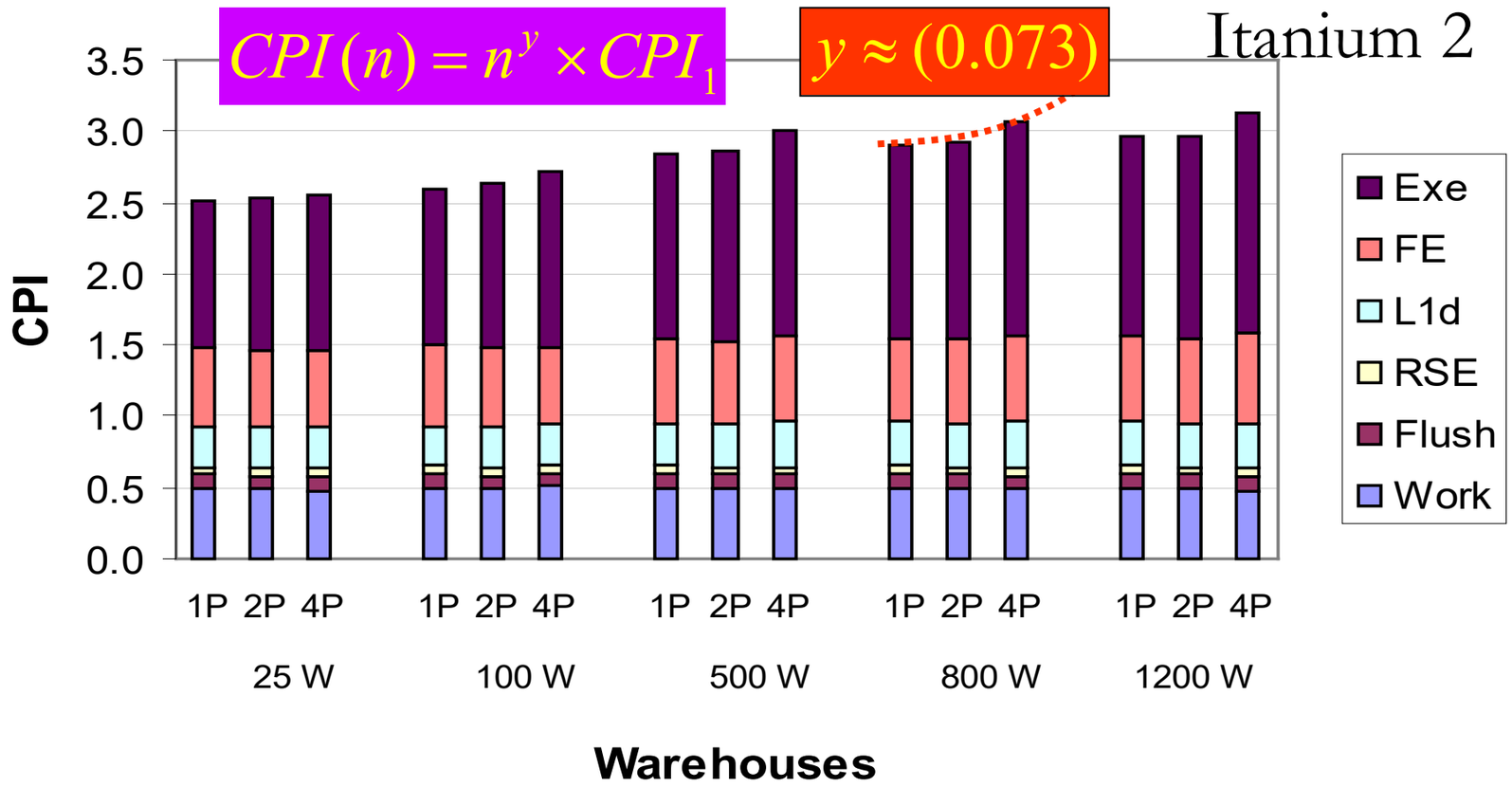
$$Speedup(n)_{MT} = \left(\frac{n}{n^x \times n^y}\right) = n^{0.833}$$

[Law #3]

[Rich Hankins & Murali Annavaram, 2004]

CPI Breakdown: Itanium 2

$$TPS = \frac{n \times Frequency}{IPX(n) \times CPI(n)}$$



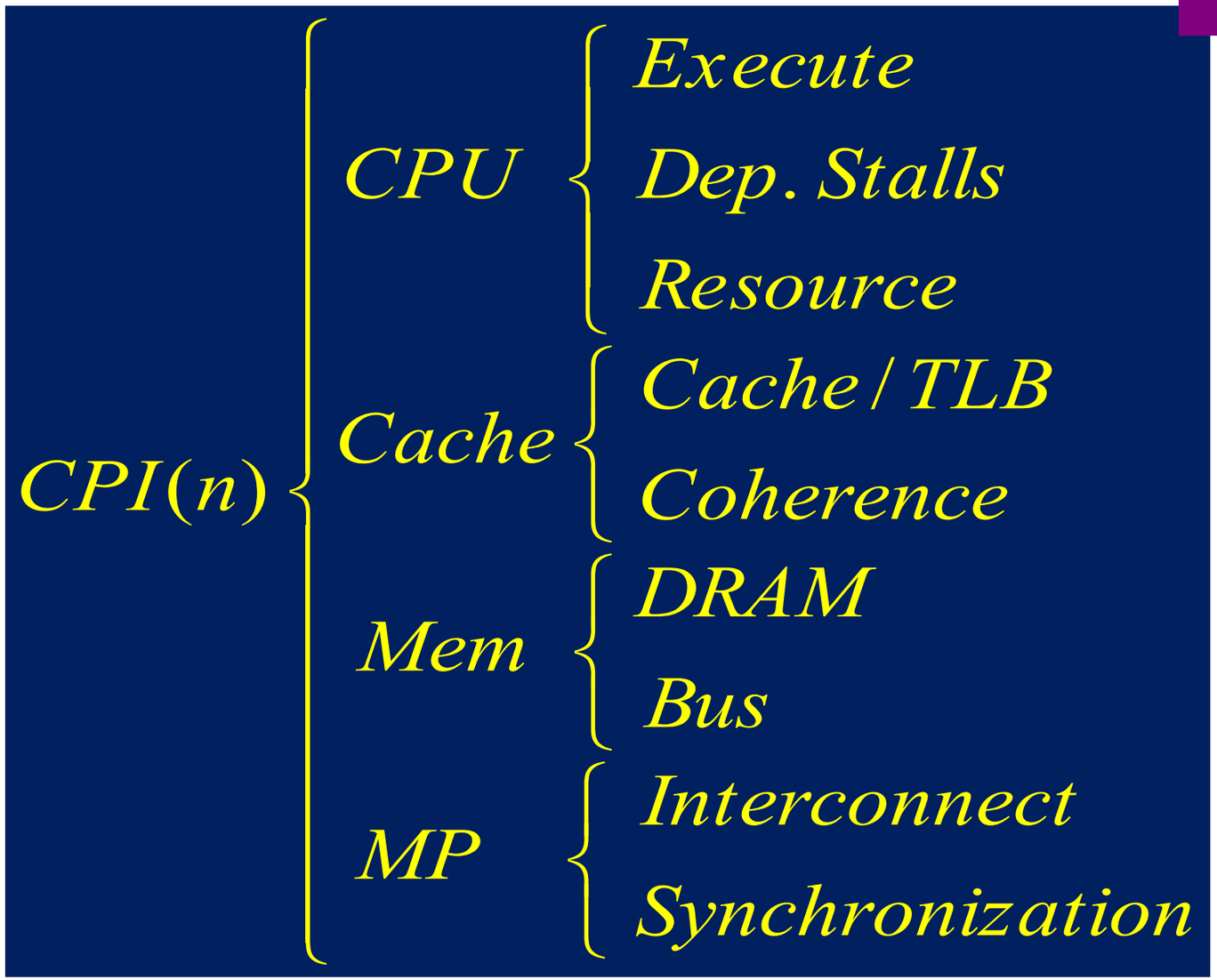
$$TPS = \frac{n \times Frequency}{\left(n^0 \times IPX_1\right) \times \left(n^{0.073} \times CPI_1\right)}$$

$$Speedup(n)_{MT} = \left(\frac{n}{n^x \times n^y}\right) = n^{0.927}$$

[Law #3]

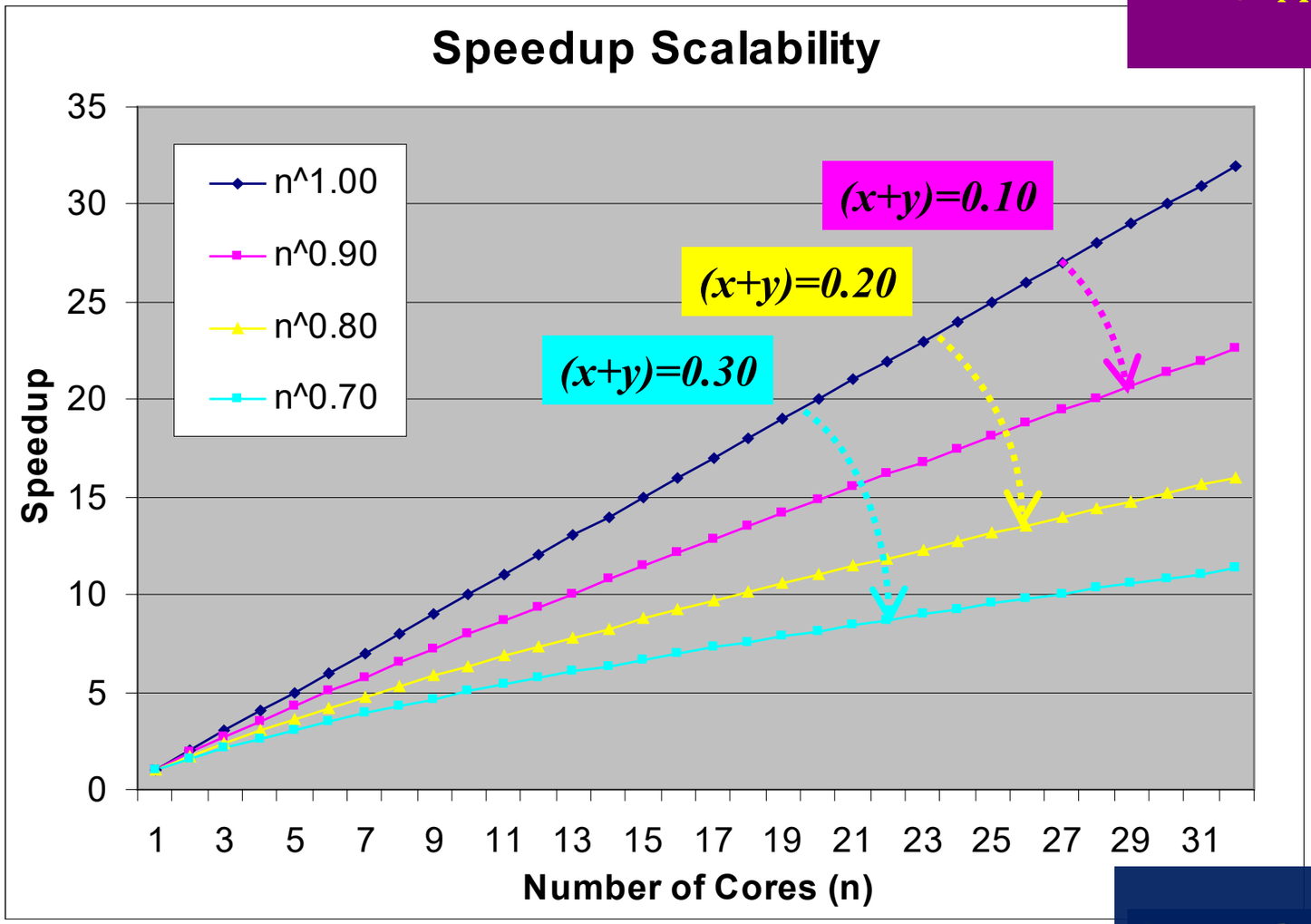
CPI Contributions

$$TPS = \frac{n \times Frequency}{IPX(n) \times CPI(n)}$$



MT Scalability Impedance

$$Perf_{MT'}(n)_{MC} = \left(\frac{n}{n^x \times n^y}\right) = n^{1-(x+y)}$$



[Carole Dulong et al., 2005]

1p-16p scaling	1-(x+y)	R^2
SEMPHY	**0.993**	0.999
PLSA	**0.963**	0.999
Rsearch	**0.931**	0.997
SVM-RFE	**0.786**	0.970
SNPs	**0.685**	0.967
GeneNet	**0.642**	0.983

$$Speedup(n)_{MT} = \left(\frac{n}{n^x \times n^y}\right) = n^{1-(x+y)}$$

Conspiring Forces Against MT Performance Scaling: Three Forms of Scalability Impedances

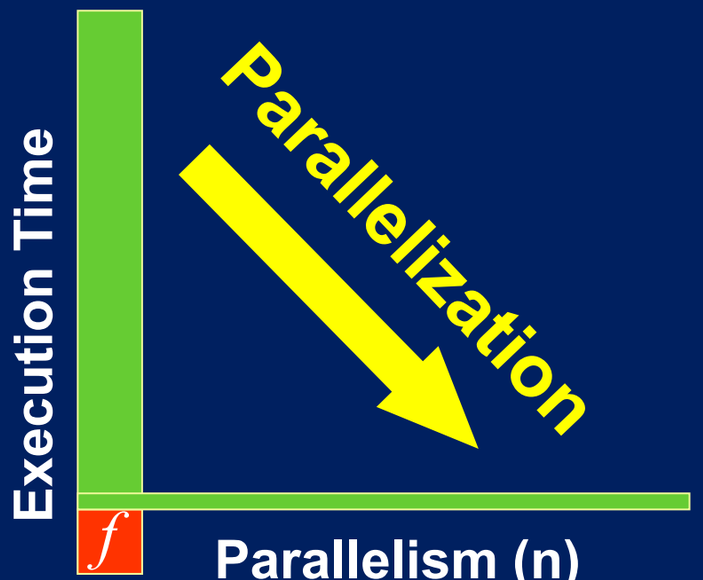
- Algorithm
  - Limitation of Language and Algorithm
  - Tyranny of Amdahl's Law (sequential bottleneck)
- Architecture
  - Increase of Path-Length Undermines Scalability
  - Increase of CPI also Undermines Scalability
- Power/Energy
  - Increased Complexity and Inefficiency of Design
  - Super-linear Power Scaling Relative to Performance

https://cis.temple.edu/~shi/wwwroot/shi/public_html/docs/amdahl/amdahl.html

Law #4 – Algorithm and Performance (Amdahl's Law & Gustafson's Law)

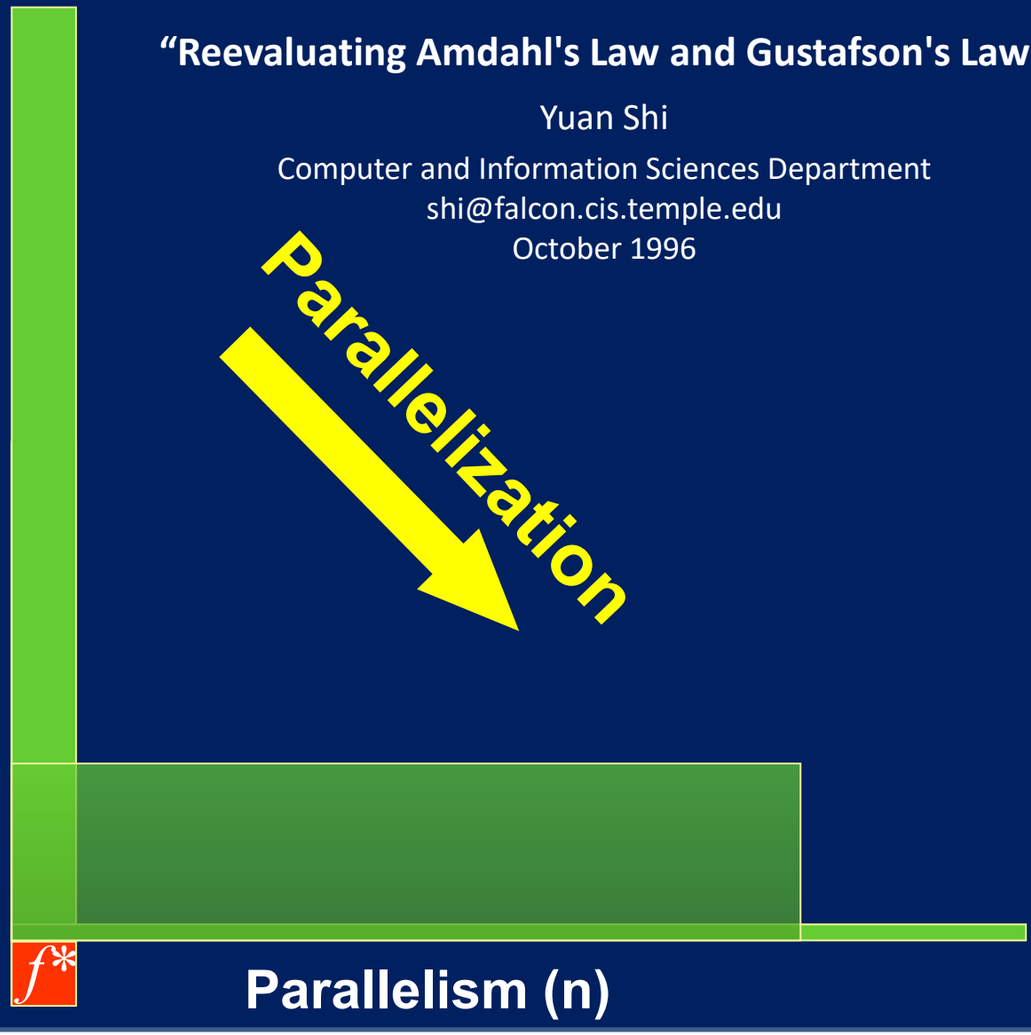
$$Speedup(n)_{ALG} = \frac{1}{\left(\frac{f}{1}\right) + \left(\frac{(1-f)}{n}\right)}$$

f = sequential %



"Reevaluating Amdahl's Law and Gustafson's Law"

Yuan Shi
Computer and Information Sciences Department
shi@falcon.cis.temple.edu
October 1996



Two Distinct & Correlated Dimensions of MT Performance Scalability (Impedance)

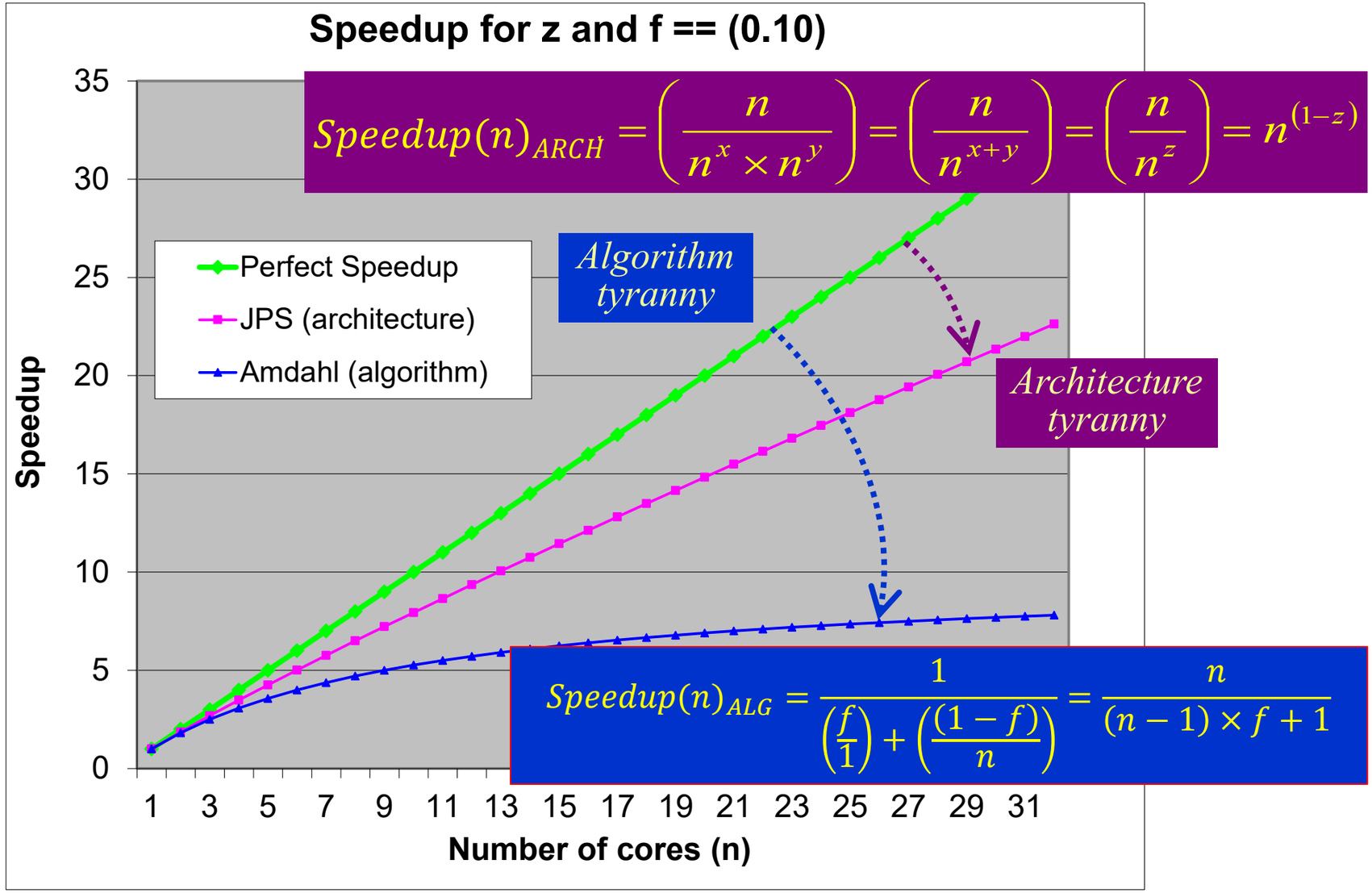
Algorithm Scalability: (Amdahl's Law)

$$Speedup(n)_{ALG} = \frac{1}{\left(\frac{f}{1}\right) + \left(\frac{(1-f)}{n}\right)} = \frac{n}{(n-1)\times f + 1}$$

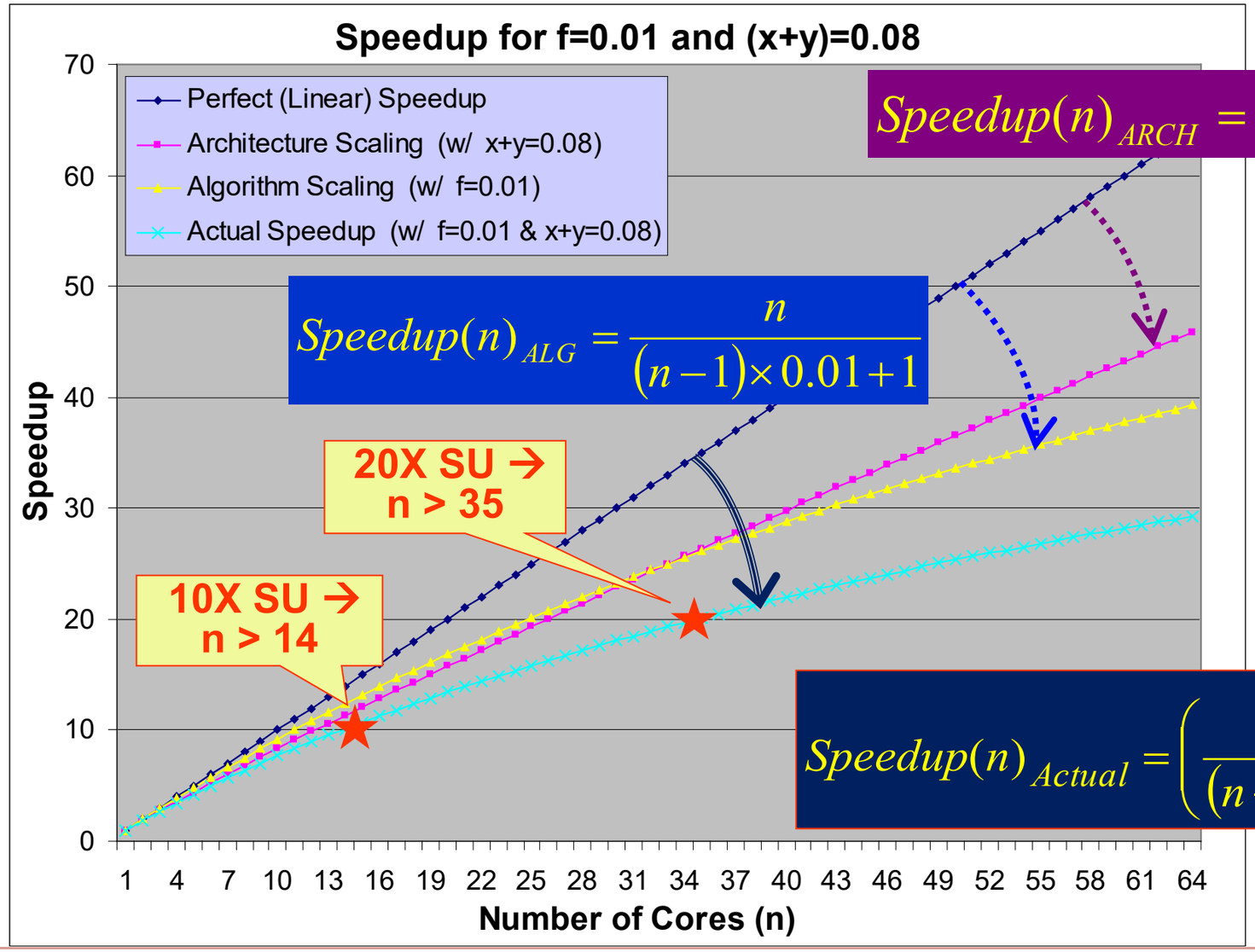
Architecture Scalability:

$$Speedup(n)_{ARCH} = \left(\frac{n}{n^x \times n^y}\right) = \left(\frac{n}{n^{x+y}}\right) = n^{1-(x+y)}$$

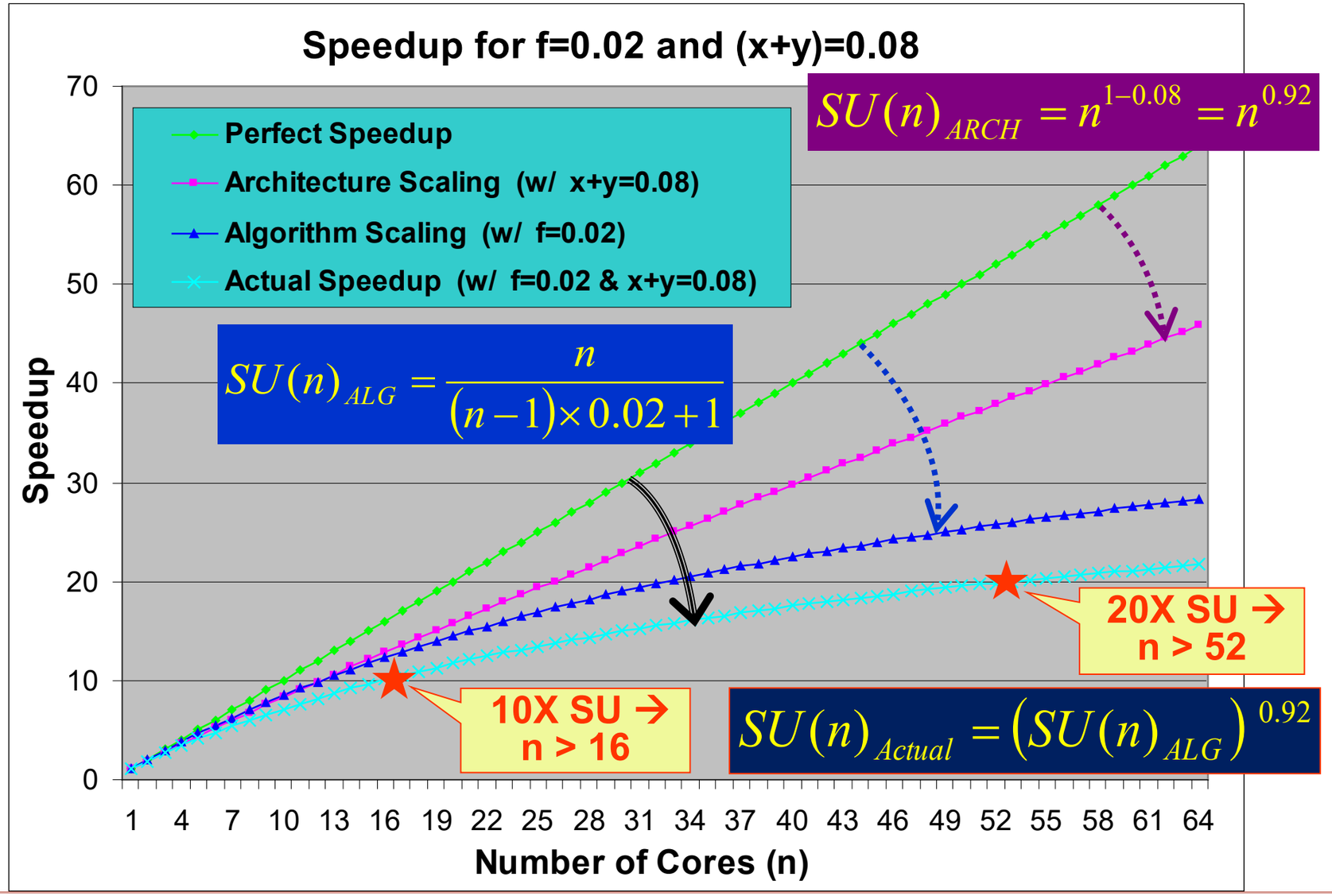
Relative (Additive) Degrees of Tyranny



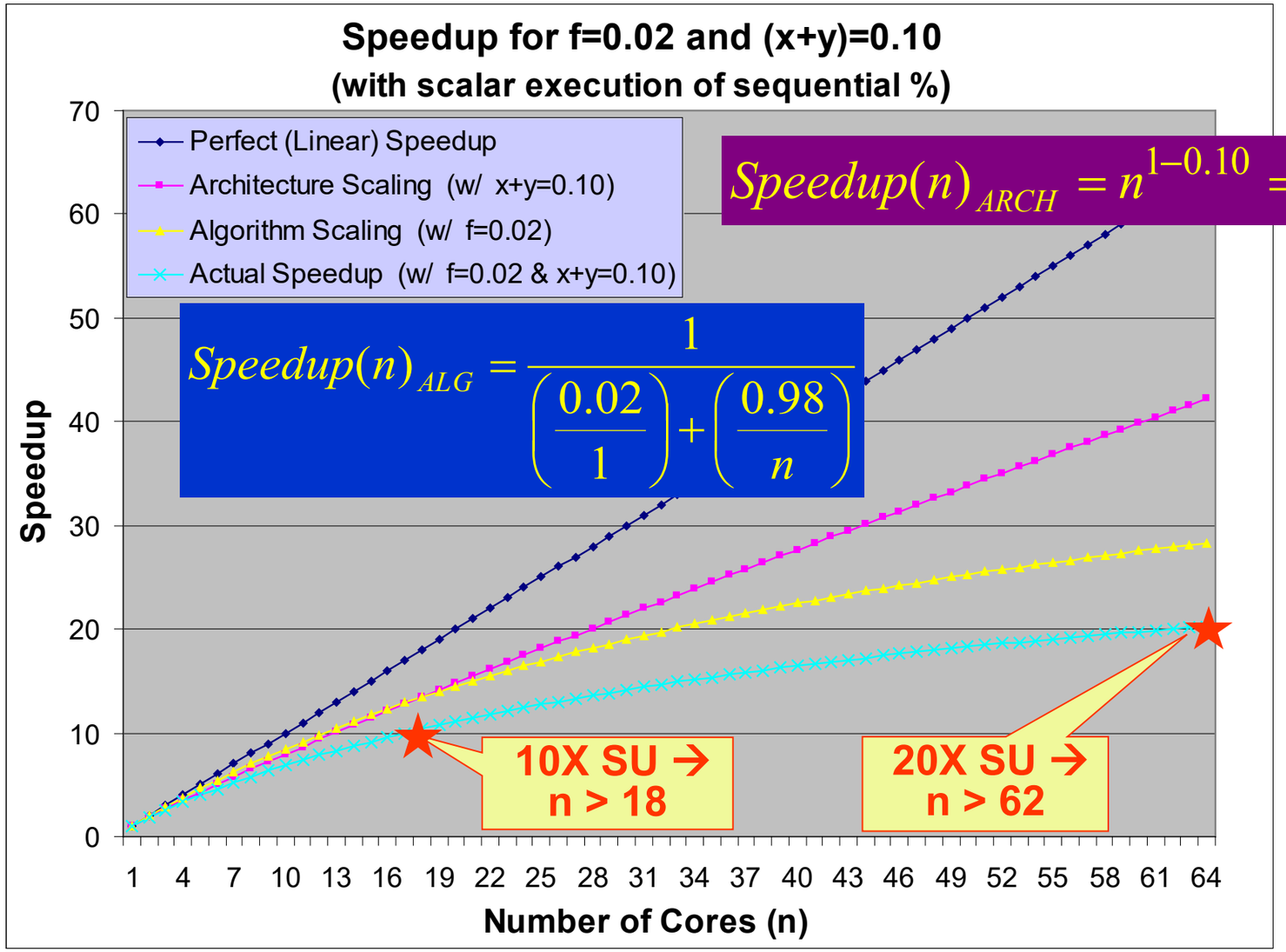
Combined Effect on Actual Speedup (f=0.01, x+y=0.08)



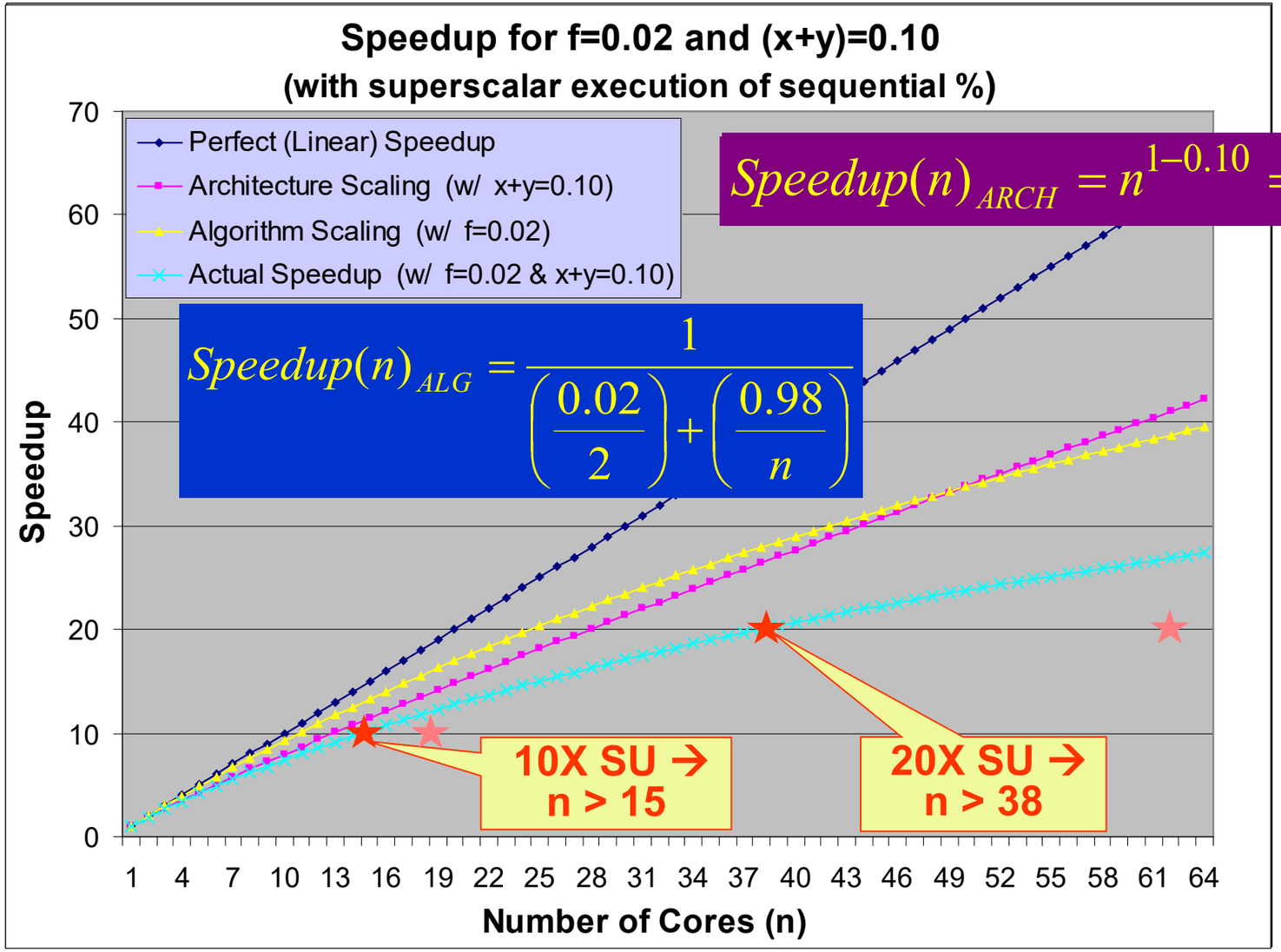
Combined Effect on Actual Speedup (f=0.02, x+y=0.08)



Combined Effect on Actual Speedup (f=0.02, x+y=0.10)



ST Performance's Impact on MT Performance



Law #5 – Performance and Power

$$Watt = \frac{Joule}{second} = \frac{Joule}{instruction} \times \frac{instruction}{cycle} \times \frac{cycle}{second}$$

$$Power = \boxed{EPI \times IPC \times Frequency}$$

$$Performance = \frac{Frequency}{PathLength \times CPI} = \frac{IPC \times Frequency}{PathLength}$$

$$Power = \boxed{EPI \times Performance \times PathLength}$$

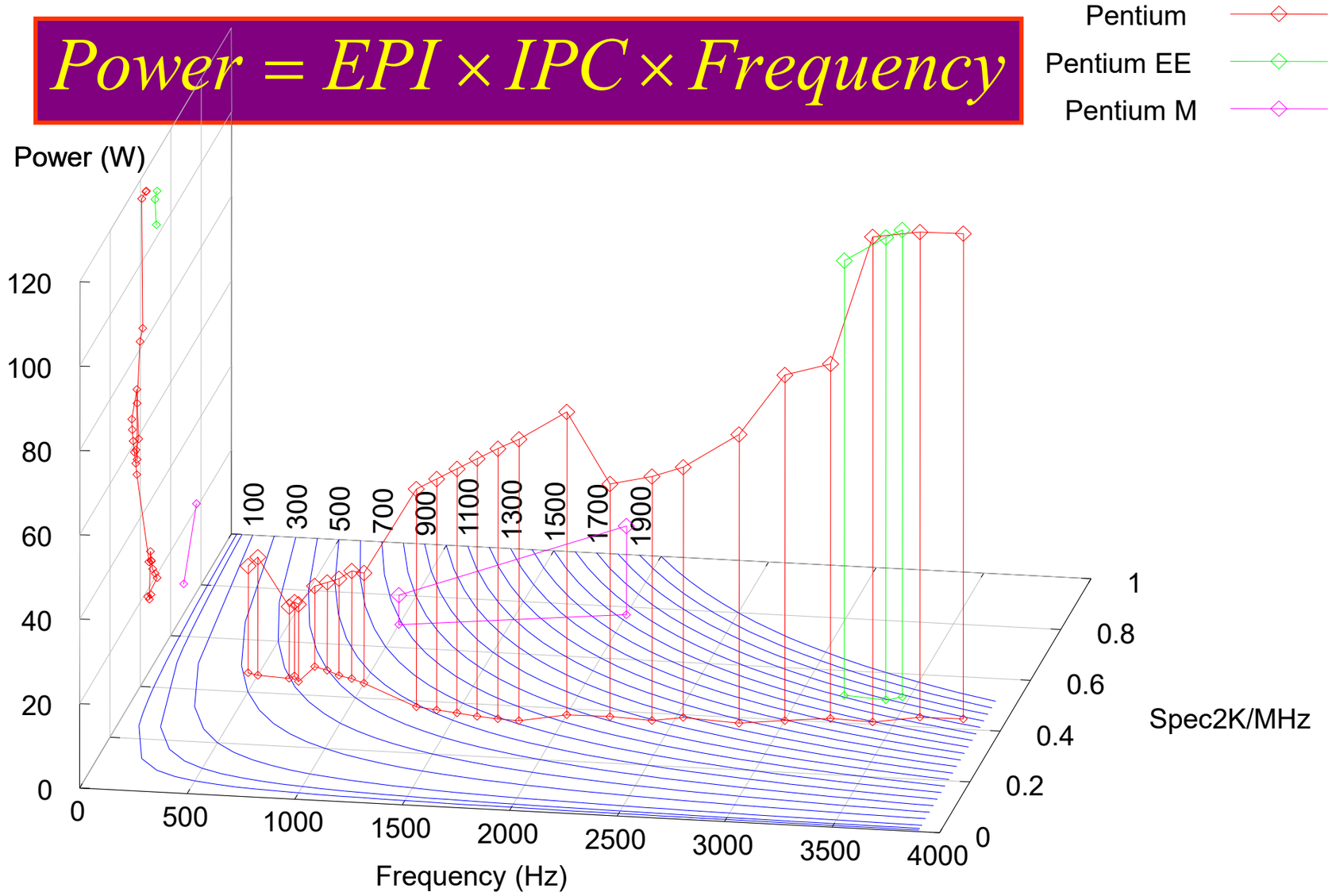
[John DeVale & Bryan Black, 2005]

Iron Law of Performance and Power

$$Power = EPI \times IPC \times Frequency$$





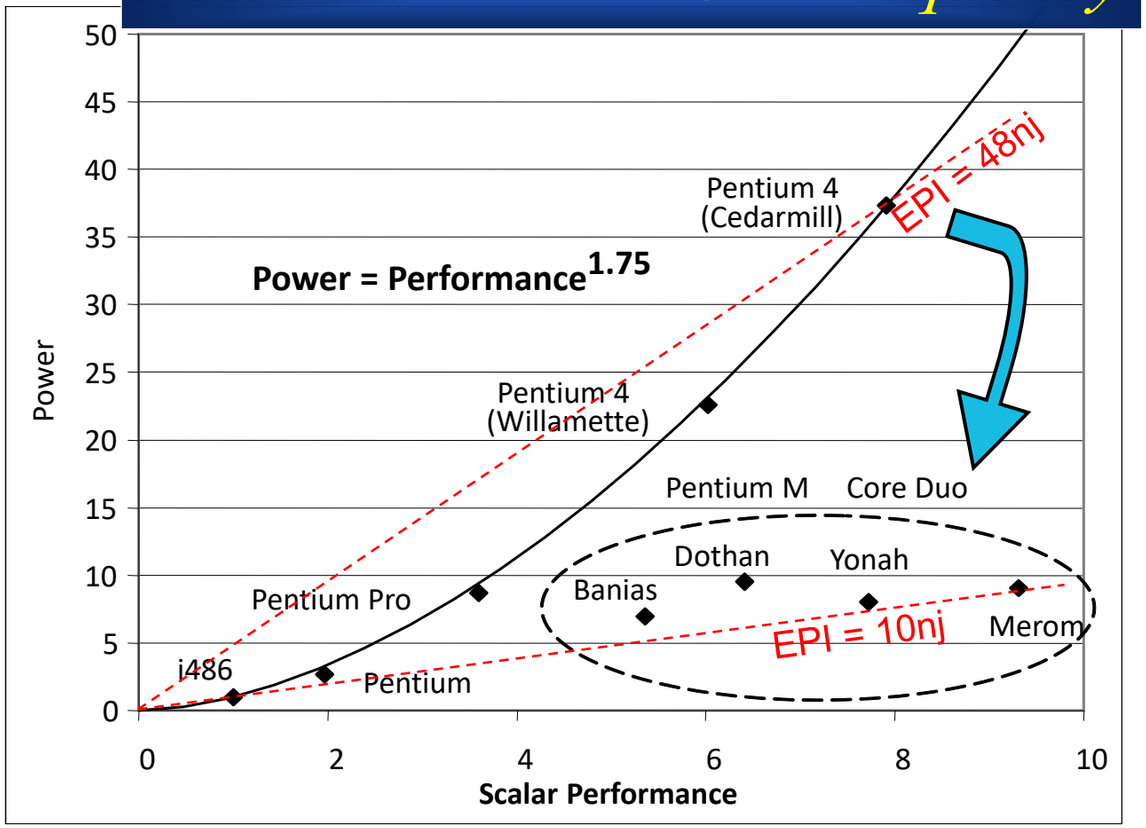


[Ed Grochowski, Intel, 2004]

Power/Performance (EPI) Evolution

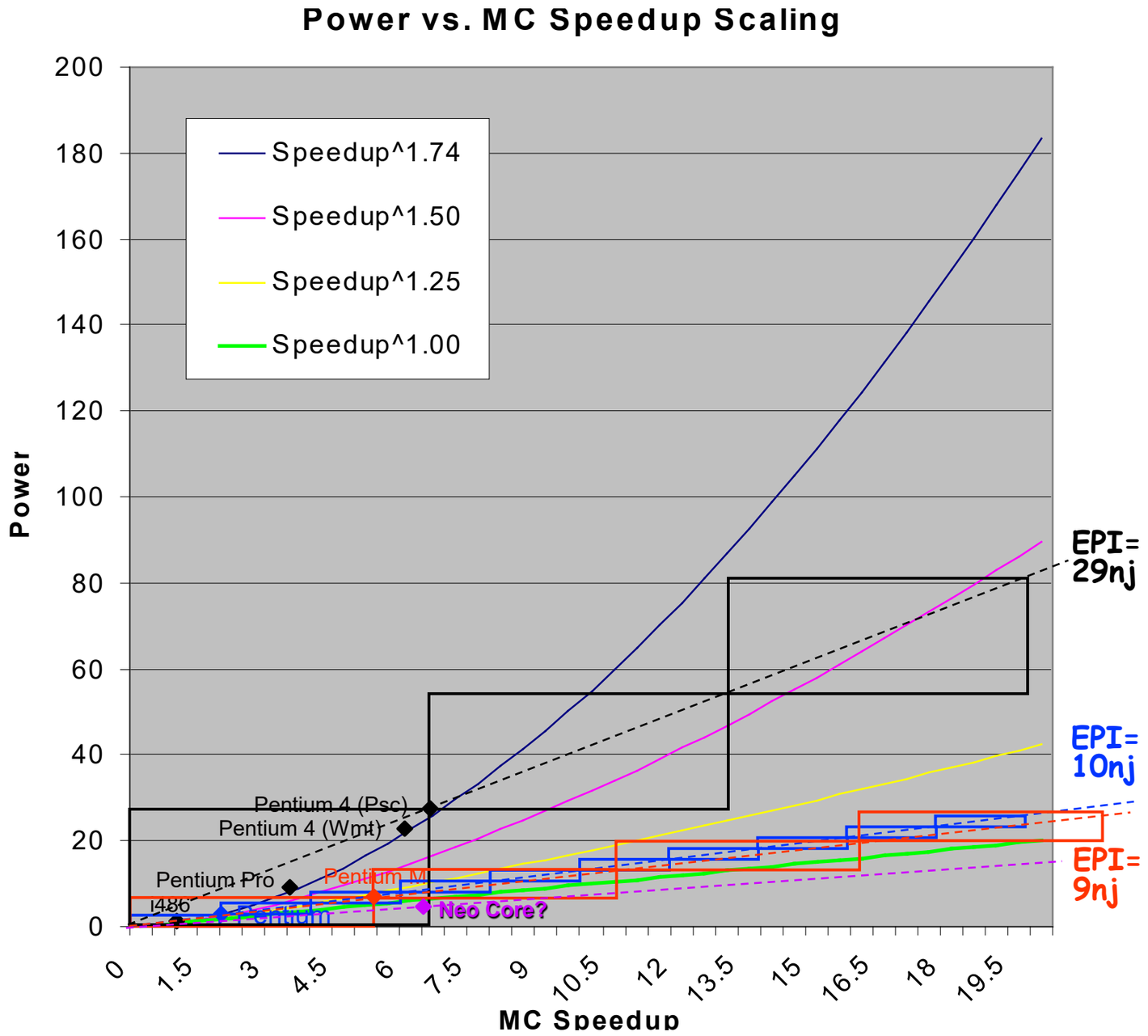


$$Power = EPI \times IPC \times Frequency$$



Intel Microprocessors	EPI (nj) 65nm at 1.33v
i486	10
Pentium	14
Pentium Pro	24
Pentium 4 (WMT)	38
Pentium 4 (CDM)	48
Pentium M (Banias)	13
Pentium M (Dothan)	15
Core Duo (Yonah)	11
Core Duo (Merom)	10

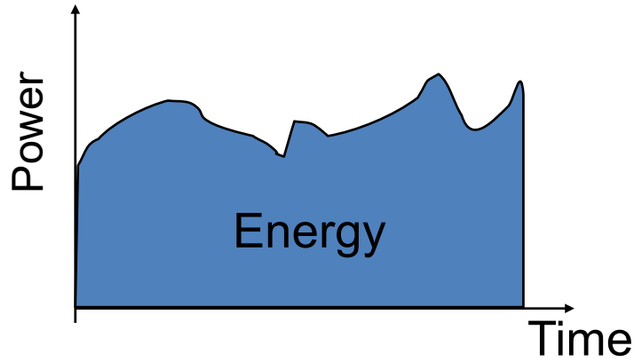
- Power: single core power (relative to i486 baseline)
- Performance: SPECint performance (relative to i486 baseline)
- EPI: average energy spent per instruction (in nano-joules)



CPU	EPI	SU
i486	7 nj	1
P5	10 nj	2
P6	17 nj	3.5
P4P-wmt	27 nj	6
P4P-psc	29 nj	6.5
Pentium M	9 nj	5.5
Neo core?	< 5 nj	6.5

- The scaling goal is not just the number of cores but maximum throughput within fixed power envelope.
- The key issue is not the power scaling of replicated cores, but the un-core power scaling that may push total power scaling towards the square law again.

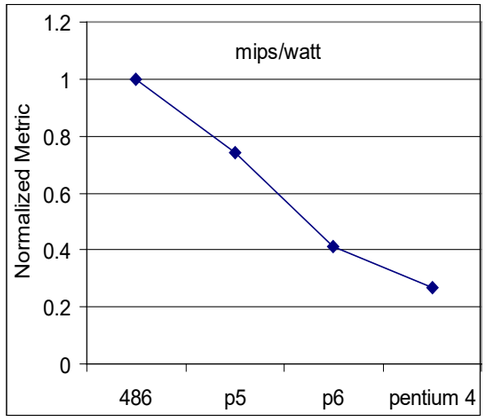
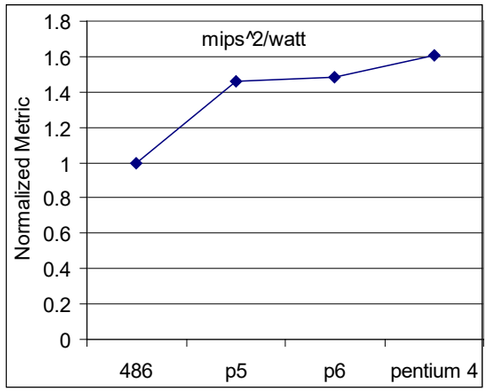
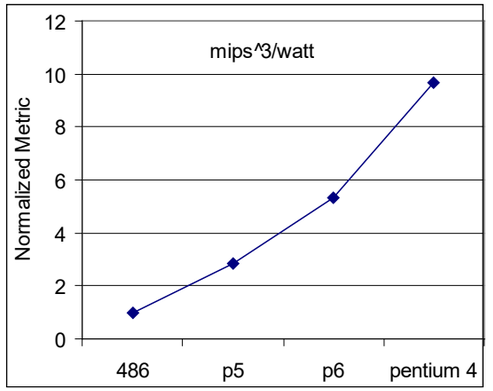
Power vs. Energy



- Energy: integral of power (area under the curve)
  - Energy & power driven by different design constraints
- With constant time base, two are “equivalent”
  - 10% reduction in power => 10% reduction in energy
- Once time changes, must treat as separate metrics
  - E.g. reduce frequency to save power => reduce performance => increase time to completion => consume more energy (perhaps)
- Power issues:
  - Power delivery (supply current @ right voltage); Thermal (don’t fry the chip); Reliability effects (chip lifetime)
- Energy issues:
  - Limited energy capacity (battery); Efficiency (work per unit energy)

Power/Energy/Performance Efficiency Metrics

- Power is good metric for deciding on the thermal envelope of the processor
- Energy is good metric in battery constrained environments
  - Task executed at ½ speed but ¼ power means ½ the energy (2T * ¼ P = ½ E)
  - 2X battery life!
- Energy*Delay (EDP) metric gives higher weight to performance
  - Same example above, ED ($(2T)^2$ * ¼ P) stays same
- Energy*$Delay^2$ gives even more weight to performance
  - Same example above shows that ½ speed is 2X worse on ED^2 metric

MIPS/watt	Common measure of power efficiency Equivalent to **energy per instruction (EPI)** Independent of time Ideal metric for throughput performance $\frac{\text{Mips}}{\text{Watt}} = \frac{\frac{\text{Instructions}}{\text{Second}}}{\frac{\text{Joules}}{\text{Second}}} = \frac{\text{Instructions}}{\text{Joule}}$	
MIPS^2/watt	Equivalent to **energy • delay (EDP)** Common metric for comparing logic families	
MIPS^3/watt	Equivalent to **energy • delay^2** Assign increasing weight to time Appropriate metric for latency performance	

Power Constrained Environment



Assumptions

- Very large chip-level multiprocessors or multicore processors are possible
- Software is multithreaded
- Power is the limiter

Goal: maintain power consumption within a fixed budget regardless of what the processor or software do

- The key to designing a microprocessor that can achieve both high scalar performance and high throughput performance is to *dynamically vary the amount of energy expended to process each instruction according to the amount of parallelism available in the software*.

P = EPI • IPS

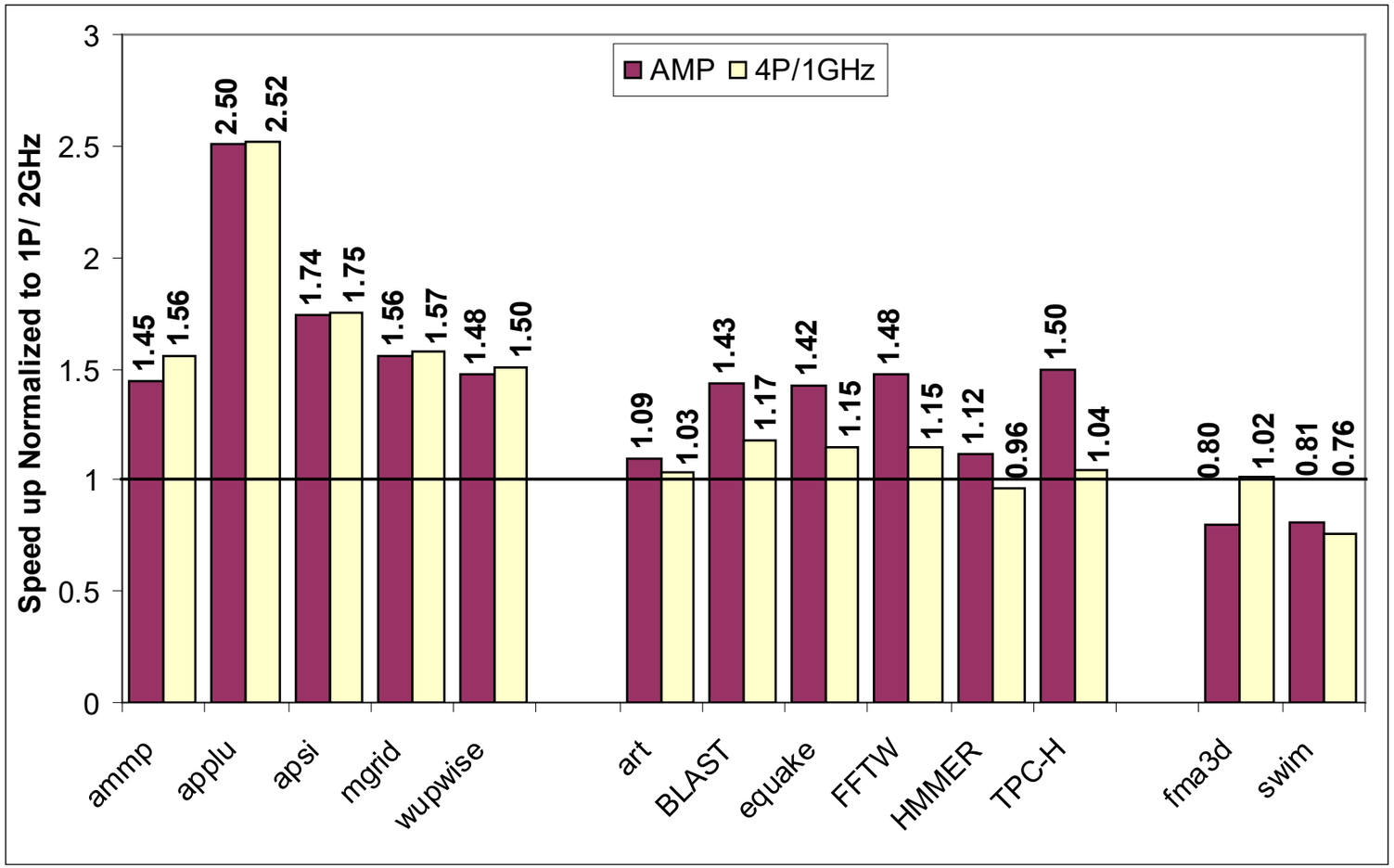
- P = fixed power budget
- EPI = energy per instruction
- IPS = aggregate number of instructions retired per second

[“Mitigating Amdahl's Law through EPI Throttling,” by Murali Annavaram, Edward Grochowski, J. P. Shen. In 32nd ISCA 2005.]

Speedup on Asymmetric MCP Prototype

Results fall into three categories

- 4P/1GHz SMP and AMP perform equally well
- AMP achieves significant speedup compared to SMP
- AMP and 4P/1GHz SMP perform worse



Why and When Asymmetric MCP is Better?

Compute percentage time in parallel and sequential

Compare run times

- AMP prototype
- Idealized AMP

3 clusters of results

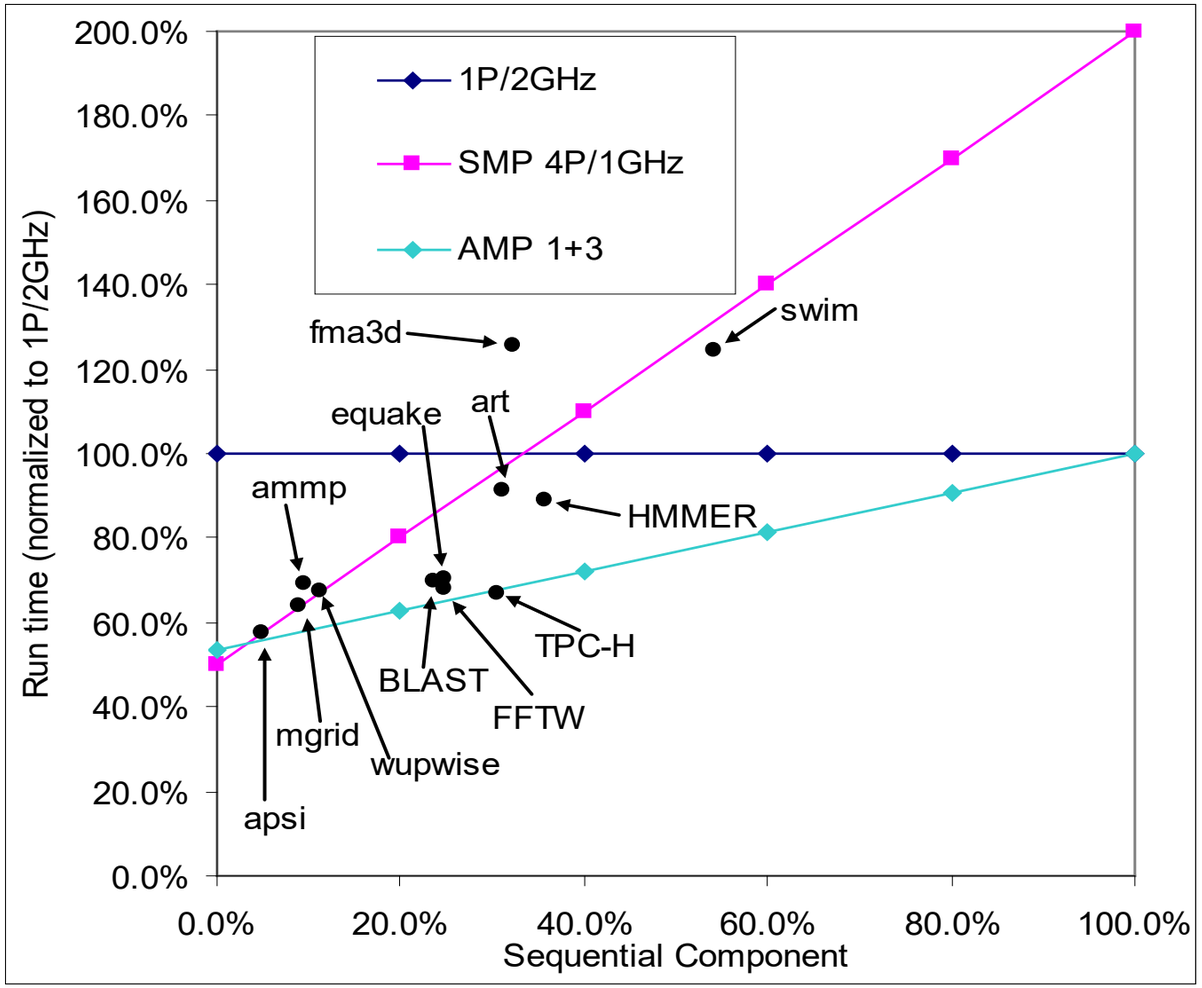
- Mostly parallel: SMP better
- Mostly sequential: 1P better
- Moderately parallel: AMP better

EPI throttling gives 38% performance increase

- Comparing AMP to 4-way SMP
- Constant power budget

Mitigated effects of Amdahl's law

- By continuously varying EPI according to the workload



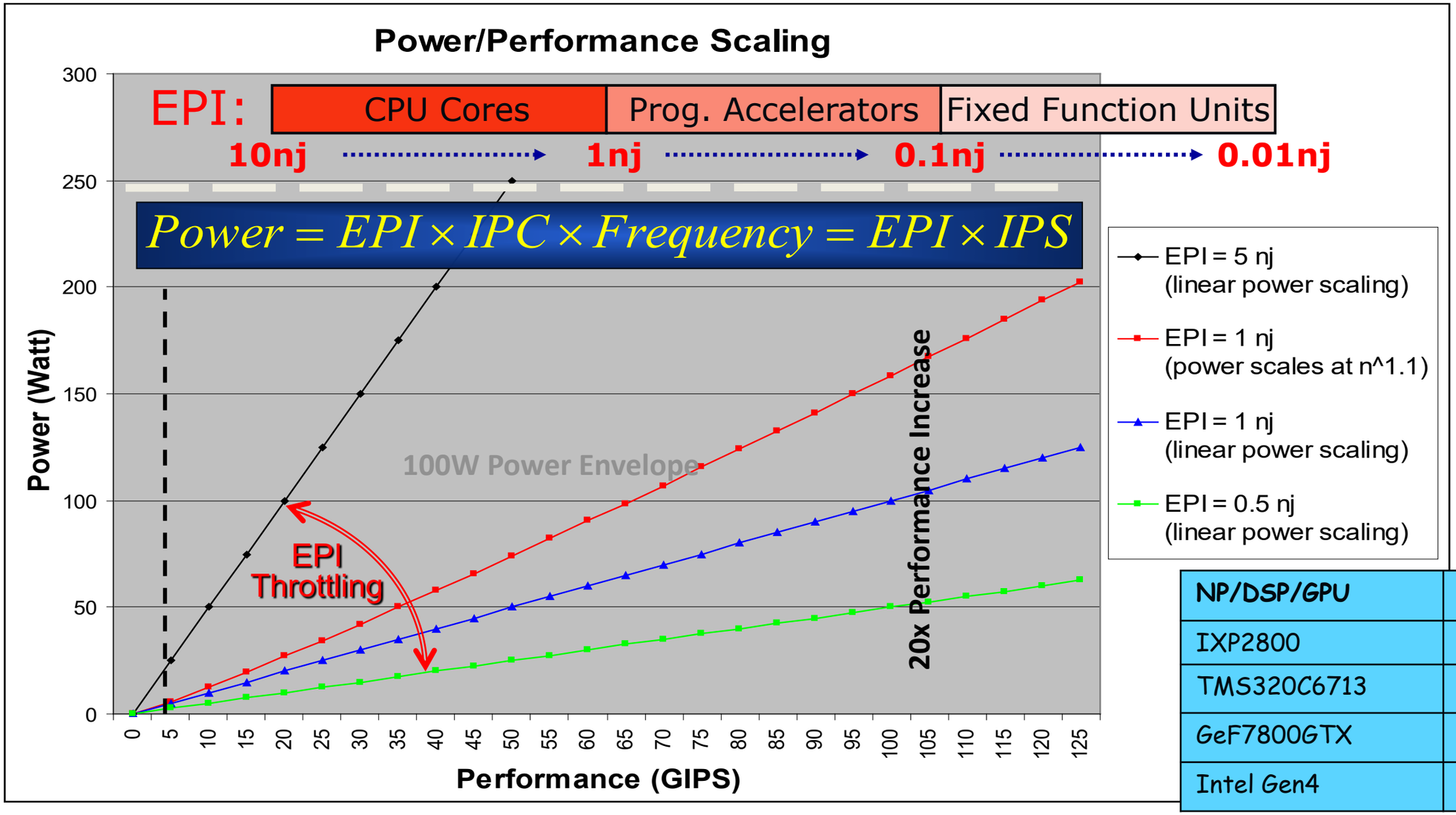
Law #6 – Power and Energy

$$Watt = \frac{Joule}{second} = \frac{Joule}{instruction} \times \frac{instruction}{cycle} \times \frac{cycle}{second}$$

$$Power = EPI \times IPC \times Frequency$$

$$Power = EPI \times IPS$$

Dynamic EPI Optimization for MCP Architectures



NP/DSP/GPU	EPI
IXP2800	~1 nj
TMS320C6713	~0.7 nj
GeF7800GTX	~0.6 nj
Intel Gen4	~0.3 nj

Energy Storage Challenge for Mobile Devices

How long between recharges?

iPad2
- 25Wh = 90kJ
- 10h use = avg 2.5W

Kindle3
- 6.5Wh = 23kJ
- "30 day use" = 30h
  avg 220mW

Typical smartphone
- 32g, 13cc, 5.5Wh = 18kJ
- +5h charging @ max 1W
- 20mW static power = 10 days standby
- 150mW notifications = 1.3 day standby
- "typical usage" 5kJ active + 13kJ standby = 1 battery charge



Key Challenges…how far can we go?



[Per Ljung, 2012]

Key Areas	Description
Active Power Saving	➢ Context Based Power Management: Offloading of communication to available connectivity, and computation to companion devices or the cloud. ➢ Workload Based Power Management: Manage power consumption based on actual usage and workload scenarios by leveraging heterogeneous cores and smart parallelism to reduce overall power.
Standby Power Saving	➢ Near-Zero-Power Standby Mode: Low-power always-on transflective bistable (TF/BS) displays; eager hibernation with instant resume. ➢ Ultra-Low-Power Always-On Device: Device with minimal standby functionality and seamless quick switch over to companion devices.
Energy Harvesting	➢ Casual Charging: Wireless inductive charging; solar charging for large surface devices. ➢ Anticipatory Preexecution: Speculative cross-device or cloud-based preprocessing and content prefetching.

Active Power Saving

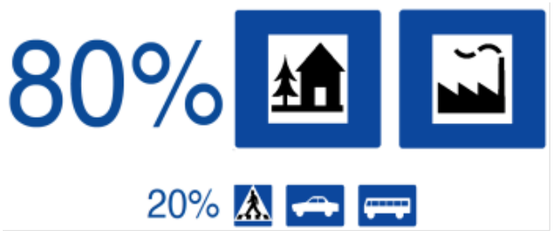
ideal 100%

1.5h @ 900mW = 1.35Wh

avg app

goal <100mW

5Wh battery: 30%



Context-Based Power Management

- Offload Communication (Local ULP radio)
  - 1m radio instead of 1000m cellular
  - Dongle, access point, femtocell box
- Offload Computation (Cross device)
  - Cloud/companion pre-compute/pre-render

80% time in home/office
40mW vs 1W

25x

trade cheap communication
for expensive computation
40mW vs 1W

25x

Workload-Based Power Management

- Power consumption based on usage scenario
  - Activity, location, history
- Approach zero energy waste
  - De-powering unused peripherals & power islands
- Heterogeneous cores & SP processor arrays (ULP)

0W vs 160mW (email, notifications)

10's mW vs 100's mW cores

Standby Power Saving

ideal 0%

22.5h @ 160mW = 3.6Wh

5Wh battery: 70%

default: email, skype

goal <10mW, 1kJ

Near-Zero-Power Standby Mode

- Zero-power always-on displays
  - Transflective, bistable
- Zero-power OS idling
  - Android, Meego, WP7 use 15mW
  - iPhone uses 5mW

TF/BS, TF/LCD, TF/OLED
5mW vs OLED 500mW

100x

better firmware 2mW vs 15mW
with hibernation 1mW vs 15mW

15x

Ultra-Low-Power Always-On Device

- Wearable accessory for notifications & voice
- Seamless quick switch over to companion
- Week-long battery life on small battery

4mW vs 200mW handset

50x

Energy Harvesting

Effectively Approach Unlimited Standby

Casual Charging

- Wireless inductive chargers — 1W desk, nightstand, car — *5h full charge*
- Instant charging for quick fix
- Solar for large surface areas — best case charge 3W for 1W tablet — *1h charge → 3h use*
- Cross-device energy sharing

Anticipatory Preexecution

- Speculative pre-processing and pre-fetching
- Cloud based or cross-device based — 40mW vs 1W — *25x*
- Context and user behavior model driven

Achievable Energy Efficiency for Mobile Devices

- Laptop (avg 10W → avg 2W)
  - Exploit large battery & connectivity
  - \- Runs offloaded apps from Phone
  - \- Gathers data for Wearable
  - \+ Display normally off
  - \+ 50 WH battery → **25 hours nonstop use**
- Smartphone (avg 200mW → avg 40mW)
  - \+ Normally hibernating in standby
  - \+ Offload apps to Laptop
  - \+ Longer battery life: 5x battery life
  - \+ 5 WH battery → **125 hours nonstop use**
- Wearable (avg 3mW)
  - \+ Always-on display
  - \+ Always fresh data feeds
  - \- Respond using Laptop or Smartphone
  - \+ Reduces avg power of Laptop & Smartphone
  - \+ 1.2 WH battery → **400 hours nonstop use**





